UME

Organization / Business Operating System

Design & Architecture Document

Document ID: UME-DA-001
Version: 1.0.0
Status: Draft — Implementation Aligned Baseline
Baseline Date: March 2026
Architecture Style: Kernel-Centric Modular Monorepo with Generated Domain Engines

1. Executive Summary

UME is a software-defined organization and business operating system (Business OS) built in Rust. It models a firm as a set of kernel-managed domain subsystems — analogous to an OS kernel managing hardware devices — where each business function is a first-class, supervised, policy-driven engine.

The system is architected as a modular monorepo with a kernel-centric composition model. Business domains (legal entity management, marketing, and an expanding enterprise function surface) run as independent crates integrated through the kernel facade. Code generation is a first-class architectural tool used to rapidly scaffold domain coverage.

This document is the authoritative design and architecture reference for the UME workspace. It consolidates the Architecture Specification (UME-ARCH-001) and the Low-Level Software Specification (UME-LLS-001) into a single coherent guide.

Key Architectural Principles

Kernel-centric supervision — ume_kernel is the composition root for all domain subsystems.
Dependency inversion — domain crates depend on ume_core contracts; they never depend on the kernel.
In-memory by default — all current persistence is process-local; repository traits abstract the boundary.
Code generation as architecture — 300+ generated modules scaffold domain breadth rapidly.
Fail-soft domain logic — missing records return rich insights/alerts rather than hard panics.

2. System Context

2.1 Mission

UME provides a complete software model of an organization. Every major enterprise function — from legal entity compliance and marketing strategy to HR, finance, GRC, and supply chain — is represented as a supervised module with well-defined lifecycle states, policy engines, and evaluation outputs.

The current baseline delivers an operational foundation: kernel bootstrapping, two fully integrated domain subsystems (Chombo and Soko), a broad generated firm-module surface (Shauri), RBAC primitives, an event bus, orchestration scaffolding, and an HTTP server for external client access.

2.2 Top-Level Capabilities

Capability	Primary Crate / Component
Kernel bootstrapping, supervision, RBAC, orchestration	ume_kernel
Shared contracts, lifecycle, error model, IDs & time	ume_core
Legal entity management & compliance filing	chombo
Marketing campaign orchestration & strategy evaluation	soko
Enterprise firm-module surface (300+ modules)	shauri_os
HTTP server for external Java and generic clients	ume_server
Interactive CLI / REPL entrypoint	ume_runtime
Java client programs (GUI + shell + watch modes)	clients/java

2.3 Codebase Scale

Crate	Approximate LOC
ume_core	131
ume_kernel	411
chombo	11,884 (45 generated policy packs + service + domain)
soko	21,659 (70 generated strategy packs + service + domain)

shauri_os	109,797 (320 generated firm modules + catalog)

3. Architectural Overview

3.1 Style and Approach

UME adopts a kernel-centric modular monorepo architecture. This style draws a deliberate analogy to OS kernel design: the kernel mediates access to domain engines (analogous to device drivers), provides shared services (event bus, RBAC, orchestration), and enforces lifecycle semantics across all subsystems.

The workspace is a single Rust Cargo workspace. Domain crates are developed independently but composed at the kernel layer. A Hybrid build system supports Rust/Cargo for the core runtime, Bazel for Java client binaries, and Maven for Java test lifecycle management.

3.2 C4 Context View (Level 1)

At the system boundary, operators, automated services, and external clients interact with UME exclusively through the runtime entrypoints — either the interactive CLI (ume_runtime) or the HTTP server (ume_server). There is no direct external access to kernel internals.

```
[Operators / Services / Java Clients / External APIs]
                      |
           +----------+----------+
           |                     |
     [ume_runtime]         [ume_server]
        (CLI/REPL)        (HTTP :8080)
           |                     |
           +----------+----------+
                      |
                [ume_kernel]
         /            |            \
    [chombo]       [soko]      [shauri_os]
         \            |            /
                [ume_core]
```

3.3 C4 Container View (Level 2)

Container	Technology	Responsibility
apps/ume_runtime	Rust binary	Interactive CLI & REPL; bootstraps kernel; emits startup readiness
apps/ume_server	Rust binary (HTTP)	REST API server; routes to kernel facade; lifecycle & health endpoints
crates/ume_kernel	Rust library	Kernel state, module registry, event bus, RBAC, orchestration, supervision

crates/ume_core	Rust library	Shared contracts: lifecycle, errors, IDs, time, metrics, models
crates/chombo	Rust library	Legal entity domain: records, policy packs, compliance evaluation, filing plans
crates/soko	Rust library	Marketing domain: campaigns, signals, strategy packs, evaluation
crates/shauri_os	Rust library	Generated firm-module catalog: 320 enterprise function modules
clients/java	Java (Bazel/Maven)	GUI and CLI client programs connecting to ume_server over HTTP

3.4 C4 Component View — Kernel

The kernel is the central composition point. Its internal components are:

```
UmeKernel
  |-- KernelConfig          (org_id, org_name, mode, audit, metrics flags)
  |-- ModuleRegistry        (Arc<RwLock<HashMap<id, Arc<dyn KernelModule>>>>)
  |-- InMemoryEventBus      (Arc<RwLock<Vec<KernelEvent>>>)
  |-- RbacEngine            (roles: HashMap, subjects: HashMap)
  |-- ChomboSubsystem       (Arc<RwLock<ChomboSubsystem>>)
  |      |-- LegalEntityService
  |      |-- InMemoryLegalEntityRepository
  |      |-- InMemoryFilingRepository
  |      +-- PolicyPacks [pack_001..pack_045]
  +-- SokoSubsystem         (Arc<RwLock<SokoSubsystem>>)
         |-- MarketingService
         |-- InMemoryCampaignRepository
         |-- InMemorySignalRepository
         +-- StrategyPacks [strategy_001..strategy_070]
```

4. Workspace & Package Topology

4.1 Directory Layout

```
ume/
├── apps/
│   ├── ume_runtime/          <- CLI/REPL entrypoint
│   └── ume_server/           <- HTTP server entrypoint
├── crates/
│   ├── ume_core/             <- shared contracts (leaf package)
│   ├── ume_kernel/           <- kernel, subsystem composition
│   ├── chombo/               <- legal entity domain
│   ├── soko/                 <- marketing domain
│   └── shauri_os/            <- enterprise firm modules
├── clients/java/             <- Java client programs
├── config/                   <- ume_server.toml
├── tools/                    <- code generation scripts (.ps1)
├── docs/
├── Cargo.toml                <- workspace root
└── Cargo.lock
```

4.2 Dependency Rules

Dependency direction is strictly enforced to preserve modularity and prevent circular dependencies:

Allowed Direction	Constraint
apps -> ume_kernel	Application binaries depend on kernel and upper-layer crates only
apps -> shauri_os	Runtime may compose with shauri_os for firm module catalog access
ume_kernel -> {ume_core, chombo, soko}	Kernel depends on core contracts and domain crates
shauri_os -> {ume_core, ume_kernel}	Firm surface uses core primitives and kernel integration points
chombo -> ume_core	Legal domain depends ONLY on shared contracts — never on kernel
soko -> ume_core	Marketing domain depends ONLY on shared contracts — never on kernel

Critical Constraint

Domain crates (chombo, soko) must NEVER depend on ume_kernel.
ume_core must remain the leaf-most shared contract package — no upstream dependencies.

Runtime app binaries must remain thin orchestration-only entrypoints.

5. Shared Core Architecture (ume_core)

ume_core is the contract foundation for the entire system. It exports a prelude of canonical shared types consumed by both the kernel and all domain crates. No business logic resides here — only primitives, contracts, and utility functions.

Module	Contents
contracts.rs	CommandEnvelope<T>, EventEnvelope<T>, Named, Versioned
domain.rs	DomainArea enum — Chombo, Soko, Marketing, Legal + others
errors.rs	UmeError (canonical error type), UmeResult<T>
lifecycle.rs	LifecycleState — Bootstrapping, Ready, Running, Degraded, Paused, Stopped, Error
ids.rs	new_id() -> String — UUID v4 generation
time.rs	now_iso8601() -> String — UTC RFC3339 timestamp
metrics.rs	MetricPoint — single telemetry observation
models.rs	KeyValue, HealthStatus — cross-domain value types
prelude.rs	Re-exports of all canonical shared symbols

5.1 Core Invariants

- All lifecycle-aware modules use LifecycleState — no local lifecycle enums in domain crates.
- All kernel-level errors normalize to UmeError — subsystem errors are mapped on the kernel facade boundary.
- IDs are string-based (UUID v4 recommended); format is not hard-enforced by type for flexibility.
- Temporal values are UTC RFC3339 strings at all API and serialization boundaries.
- All monetary amounts use integer minor units (e.g., *_cents fields) — no floating-point money.

6. Kernel Architecture (ume_kernel)

The kernel is the single composition root. It owns the runtime state of all domain subsystems and exposes a unified facade for all domain operations. All external callers interact with domain capabilities exclusively through kernel facade methods.

6.1 Kernel Configuration

Field	Default Value
org_id	org-default
org_name	Default Firm
mode	development
enable_audit	true
enable_metrics	true

6.2 Bootstrap Sequence

The kernel boot path is deterministic and synchronous in the current baseline:

1. Instantiate UmeKernel::new(config)
2. Initialize ModuleRegistry, InMemoryEventBus, RbacEngine
3. Instantiate ChomboSubsystem::default() — loads LegalEntityService + all 45 policy packs
4. Instantiate SokoSubsystem::default() — loads MarketingService + all 70 strategy packs
5. Accept org bootstrap via bootstrap_org(org_name, org_type, region)
6. Boot ShauriApp — instantiates all 320 firm modules
7. Emit startup status line: booted org={org_id} modules={n} chombo_packs={n} soko_packs={n}

The org_id generated by bootstrap_org follows a deterministic naming convention:

```
  org_id = "org-{org_type_lower}-{region_lower}"
  Example: bootstrap_org("Acme Corp", "enterprise", "us-east") ->
"org-enterprise-us-east"
```

6.3 Module Runtime Contract

All kernel-managed modules implement the KernelModule trait:

```
pub trait KernelModule: Send + Sync {
    fn id(&self) -> &'static str;
    fn domain(&self) -> DomainArea;
    fn state(&self) -> LifecycleState;
    fn start(&self) -> UmeResult<()>;
```

```
    fn stop(&self) -> UmeResult<()>;
    fn describe(&self) -> String;
}
```

The ModuleRegistry enforces unique module IDs. Duplicate registration returns UmeError::ModuleAlreadyExists. Lookup of an unknown ID returns UmeError::ModuleNotFound.

6.4 Event Bus

The InMemoryEventBus provides append-only in-process event streaming. All domain mutations should emit KernelEvents for auditability.

KernelEvent Field	Type / Description
topic	String — normative topic names: e.g. chombo.entity.evaluated
source	String — emitting component identifier
payload	String — JSON-encoded event body (typed envelopes in future revisions)
at	String — UTC RFC3339 timestamp

6.5 RBAC Engine

The RbacEngine implements a subject-role-permission model for authorization. Permission resolution is the union of all permissions across a subject's assigned roles.

Concept	Model
Role	Role { name: String, permissions: Vec<String> }
Subject	Subject { id: String, roles: Vec<String> }
Permission format	{domain}.{resource}.{action} — e.g. chombo.entity.write
Unknown subject	Returns false (deny by default)
Unknown role reference	Silently ignored (permissive forward-compat)
can() API	can(subject_id, permission) -> UmeResult<bool>

Required permissions for currently implemented subsystems:

- chombo.entity.write — create/update legal entities
- chombo.entity.evaluate — evaluate entity compliance
- soko.campaign.write — create/update campaigns

- soko.signal.ingest — ingest market signals
- soko.campaign.evaluate — evaluate campaign strategy

6.6 Supervision

The supervisor uses a policy-driven decision model. The default policy is: max_restarts = 3, backoff_ms = 1000, restart_on_failure = true.

Module State	Supervisor Decision
Error (restart enabled)	restart
Degraded	observe
Stopped	manual
Other states	noop

6.7 Orchestration

The orchestrator provides workflow definition validation and summarization. Workflows are composed of typed steps with per-step action types and timeout configurations:

```
WorkflowDefinition { id, domain, steps: Vec<WorkflowStep> }
WorkflowStep { id, name, action, timeout_ms }
```

6.8 Kernel Public Facade

All domain access flows through the kernel facade. The following methods are the normative external contract:

```
// Organization
pub fn bootstrap_org(&self, org_name, org_type, region) -> String

// Chombo (Legal Entity)
pub fn chombo_policy_pack_count(&self) -> usize
pub fn chombo_register_entity(&self, entity: LegalEntityRecord)
pub fn chombo_evaluate_entity(&self, entity_id: &str) -> ChomboEvaluation

// Soko (Marketing)
pub fn soko_strategy_pack_count(&self) -> usize
pub fn soko_summary(&self) -> SokoSummary
pub fn soko_register_campaign(&self, campaign: MarketingCampaign)
pub fn soko_ingest_signal(&self, signal: MarketSignal)
pub fn soko_evaluate_campaign(&self, campaign_id: &str) -> CampaignEvaluation
```

7. Legal Entity Domain (chombo)

Chombo is the legal entity management subsystem. It provides legal entity registration, compliance policy evaluation, and filing plan synthesis. Compliance checks are executed by a fleet of policy packs, which run independently and aggregate their results.

7.1 Domain Models

Model	Purpose
LegalEntityRecord	Primary aggregate — entity master record
EntityMetadata	Supplemental metadata attached to an entity
ComplianceAlert	Alert output from policy pack evaluation
FilingRecord	Filing plan entry generated by policy packs
RelationshipRecord	Inter-entity relationship (parent/subsidiary/etc.)
GovernanceProfile	Board structure and governance attributes
JurisdictionPolicy	Jurisdiction-specific policy parameters
EntitySearchQuery / Result	Search and filter models for entity lookup

7.2 Commands & Events

Commands	Events
RegisterLegalEntityCommand	LegalEntityRegisteredEvent
UpdateLegalEntityCommand	LegalEntityUpdatedEvent
DeactivateLegalEntityCommand	ComplianceAlertEvent
FieldUpdate	FilingScheduledEvent

7.3 Policy Engine

The policy pack fleet (pack_001 through pack_045, built by build_default_policy_packs()) drives all compliance evaluation. Each pack implements the LegalPolicyPack trait:

```
pub trait LegalPolicyPack {
    fn id(&self) -> &'static str;
    fn evaluate(&self, entity: &LegalEntityRecord) -> Vec<ComplianceAlert>;
    fn build_filings(&self, entity: &LegalEntityRecord) -> Vec<FilingRecord>;
}
```

Each pack performs at minimum: name validation (ENTITY-NAME-REQUIRED) and registration number validation (REGISTRATION-NUMBER-REQUIRED), then emits synthetic rule alerts (PACKxxx-RULEyyy) and builds filing plans.

7.4 Evaluation Sequence

```
Caller -> kernel.chombo_evaluate_entity(entity_id)
       -> ChomboSubsystem.evaluate_entity(entity_id)
       -> LegalEntityService.evaluate_entity(entity_id)
            -> repository.get_entity(entity_id)
            -> for each LegalPolicyPack: evaluate(entity) -> alerts
       -> LegalEntityService.schedule_filings(entity_id)
            -> for each LegalPolicyPack: build_filings(entity)
            -> filing repository upsert
       -> return ChomboEvaluation { alerts, filings }
```

Soft-Failure Behavior

If entity_id is not found in the repository, Chombo returns a synthetic high-severity ComplianceAlert with code ENTITY-NOT-FOUND rather than panicking or returning an error. The evaluation output remains structurally valid for downstream consumers.

7.5 Default Workflow

The EntityWorkflow defines the default registration lifecycle: collect-docs -> run-kyc -> register. An action triage utility (triage(alerts, filings, entity)) converts evaluation outputs into prioritized action strings for UI presentation.

8. Marketing Domain (soko)

Soko implements marketing campaign orchestration. It handles campaign registration, real-time market signal ingestion, multi-strategy evaluation, and activation plan synthesis. Like Chombo, it uses a policy-pack-style fleet of strategy modules for analytical execution.

8.1 Domain Models

Model	Purpose
MarketingCampaign	Primary aggregate — campaign record
MarketSignal	Telemetry signal ingested against a campaign
StrategyInsight	Analytical insight produced by a strategy pack
ActivationAction	Concrete action recommendation from strategy evaluation
CampaignEvaluation	Evaluation output: insights + actions
ChannelAllocation	Budget and channel distribution model
AudiencePersona	Audience targeting model
ContentAsset	Creative asset metadata

8.2 Strategy Engine

The strategy pack fleet (strategy_001 through strategy_070, built by build_default_strategy_packs()) drives all campaign evaluation. Each pack implements the MarketingStrategyPack trait:

```
pub trait MarketingStrategyPack {
    fn id(&self) -> &'static str;
    fn evaluate(&self, campaign: &MarketingCampaign,
                signals: &[MarketSignal]) -> Vec<StrategyInsight>;
    fn plan_actions(&self, campaign: &MarketingCampaign,
                    signals: &[MarketSignal]) -> Vec<ActivationAction>;
}
```

8.3 Evaluation Sequence

```
Caller -> kernel.soko_evaluate_campaign(campaign_id)
       -> SokoSubsystem.evaluate_campaign(campaign_id)
       -> MarketingService.evaluate_campaign(campaign_id)
            -> campaign repository get_campaign
            -> signal repository list_signals_for(campaign_id)
            -> for each MarketingStrategyPack:
                 evaluate(campaign, signals) -> insights
                 plan_actions(campaign, signals) -> actions
```

```
        -> return CampaignEvaluation { insights, actions }
```

8.4 Default Workflow

The MarketingWorkflow defines four lifecycle phases: audience-discovery -> creative-assembly -> launch -> optimization. The build_execution_notes utility converts evaluation outputs into annotated execution guidance strings.

9. Enterprise Firm Surface (shauri_os)

shauri_os hosts the broad enterprise function module catalog. It is currently a generated surface of 320 structurally valid firm modules spanning all major business function domains. Each module provides a consistent lifecycle interface and is a scaffold for progressive replacement with domain-specific logic.

9.1 FirmModule Contract

```
pub trait FirmModule {
    fn id(&self) -> &'static str;
    fn configure(&mut self, cfg: ModuleConfig);
    fn run_cycle(&mut self) -> ModuleReport;
}
```

Data Structure	Fields
ModuleConfig	id, enabled, owner_team, sla_tier
ModuleReport	id, healthy: bool, notes: Vec<String>

9.2 Coverage Domains

The 320 generated modules (mod_001_* through mod_320_*) cover the following enterprise function clusters:

- Finance, accounting, treasury, investment management
- HR, talent acquisition, performance management, learning management
- Legal entity, GRC, risk management, ESG/CSR, compliance
- IT, enterprise security, backup & recovery
- Marketing, CRM, sales, branding, PR
- Supply chain, procurement, logistics, inventory, warehouse
- Workflow automation, orchestration, BPM, scheduling
- Analytics, dashboards, KPIs, master data management
- Knowledge management, content management, digital workspace
- Portfolio, program, project management
- API integration, extensibility, digital twin placeholders

9.3 Catalog Boot

```
ShauriApp::boot() -> usize
  -> build_module_catalog() -> Vec<Box<dyn FirmModule>>
  -> Returns count of successfully instantiated modules
```

9.4 Generated Module Behavioral Pattern

Each generated module maintains local state (cfg, cycle counter, ledger), computes a synthetic score from the cycle value and configuration parameters, appends synthetic control notes, and returns health as: enabled && score >= 0. This pattern ensures structural consistency and enables load/performance characterization until real domain logic is introduced.

10. Data & State Architecture

10.1 Current In-Memory State Holders

Component	State Held
Kernel / ModuleRegistry	HashMap of module ID -> Arc<dyn KernelModule>
Kernel / InMemoryEventBus	Vec<KernelEvent> — append-only event log
Kernel / RbacEngine	HashMap<name, Role>, HashMap<id, Subject>
Chombo	HashMap of entity ID -> LegalEntityRecord; HashMap of entity ID -> Vec<FilingRecord>
Soko	HashMap of campaign ID -> MarketingCampaign; HashMap of campaign ID -> Vec<MarketSignal>
Shauri	Per-module: ModuleConfig, cycle counter, Vec<String> ledger

10.2 Persistence Model

Current Status: In-Memory Only

All state is process-local and non-durable. No data survives a process restart. This is appropriate for rapid prototyping and deterministic behavior, but unsuitable for production state retention.

The repository trait pattern provides the abstraction boundary for future persistent adapters. Adding persistence does not require changing kernel facades — only new adapter implementations of the repository traits.

10.3 Target Logical Schema (Future Persistent Backend)

Table	Bounded Context
legal_entities	Chombo — entity master records
legal_filings	Chombo — filing plans and outcomes
legal_relationships	Chombo — inter-entity relationship graph
marketing_campaigns	Soko — campaign master records
marketing_signals	Soko — signal telemetry stream
kernel_events	Kernel — event bus append log
rbac_roles	Kernel — role definitions

rbac_subject_roles	Kernel — subject-role assignments
module_config_snapshots	Shauri — module configuration snapshots
module_execution_reports	Shauri — module cycle execution reports

10.4 Serialization Rules

- All external command/event models derive: Serialize, Deserialize, Debug, Clone.
- Field naming convention: snake_case throughout.
- Monetary amounts: integer minor units (*_cents fields) — no floating-point money.
- Timestamps: UTC RFC3339 strings at all API boundaries.
- IDs: String type; UUID v4 format recommended.
- Do not serialize runtime locks (Arc<RwLock<...>>) in subsystem structs.

11. Code Generation Architecture

Code generation is a first-class architectural approach in UME. The three primary generators produce the policy pack fleet, strategy pack fleet, and enterprise module catalog respectively. Generated code follows deterministic naming and structural conventions.

11.1 Generators

Generator	Output Target	Output Volume
tools/generate_ume.ps1	Base workspace + shauri_os module catalog	320 firm module files
tools/generate_chombo.ps1	Chombo policy pack fleet	45 policy pack files + mod.rs
tools/generate_soko.ps1	Soko strategy pack fleet	70 strategy pack files + mod.rs

11.2 Naming Conventions

- Policy packs: pack_001 through pack_045
- Strategy packs: strategy_001 through strategy_070
- Firm modules: mod_001_{domain} through mod_320_{domain}

11.3 Architectural Tradeoffs

Benefits	Costs
Massive domain surface rapidly available	Large compile-time footprint
Uniform interfaces and behavior across all packs/modules	Synthetic behavior dominates functional semantics in early phases
Clear scaffold for progressive replacement with real logic	Harder manual navigation due to module count
Predictable structural repetition for load/performance testing	Generated code volume may mask regressions without tooling

12. Concurrency & Thread Safety

UME uses parking_lot::RwLock and Arc<T> for all shared mutable state. Subsystem instances are held under independent locks in the kernel, allowing concurrent read access to separate domains without contention.

12.1 Locking Model

Operation Class	Lock Behavior
Read-only metrics / summary queries	Acquire read lock on the relevant subsystem only
Mutating domain operations (register, ingest)	Acquire write lock on the relevant subsystem only
Cross-subsystem operations	At most one write lock per call — nested write locks are prohibited
Event bus publish/read_all	Independent Arc<RwLock<Vec<KernelEvent>>> — no subsystem lock

12.2 Current Constraints

- Lock granularity is coarse at the subsystem level — one lock per subsystem instance.
- No lock ordering framework is defined for hypothetical future multi-subsystem operations.
- No async scheduling in the current implementation — all execution is synchronous.
- Lock hold duration is bounded to in-memory operation windows (no I/O inside locked sections).

12.3 Performance Targets (In-Memory Baseline)

Metric	Target
Evaluation latency (p50)	< 30 ms
Evaluation latency (p95)	< 100 ms
Per-call heap growth	Bounded — no unbounded allocation outside generated insight/action vectors
Repository list operations	O(n) linear in collection size

13. Error Handling & Fault Model

Error Class	Maps To / Trigger

ModuleNotFound	Registry lookup for unknown module ID
ModuleAlreadyExists	Duplicate module ID registration attempt
InvalidOperation	Domain validation failures
AuthorizationFailed	RBAC permission deny
Serialization	JSON serialize/deserialize failures
Runtime	Unexpected runtime failures

Domain subsystems use a soft-failure pattern for missing records: a missing entity or campaign returns a structured high-severity alert/insight rather than a hard error, keeping evaluation outputs structurally valid for all downstream consumers.

14. Security Architecture

14.1 Current Controls

Component	Status	Notes
In-memory RBAC model	LIVE	Role/permission model implemented in RbacEngine
Permission namespace (domain.resource.action)	LIVE	Normative permission format defined
RBAC enforcement on kernel facade methods	GAP	Chombo/Soko kernel methods do not yet enforce RBAC
Authentication boundary	GAP	No authn layer in ume_runtime or ume_server currently
Encryption model	GAP	No encryption specified in this revision
Audit event emission	PARTIAL	Event bus in place; domain calls not yet publishing events

14.2 Required Next Controls

- Add permission guard wrappers for all public kernel domain facade methods.
- Add authentication boundary at the ume_server API gateway layer.
- Enforce RBAC before every mutate and evaluate call in Chombo and Soko paths.
- Add structured RBAC denial events to the event bus (topic: security.rbac.denied).

15. Observability Architecture

15.1 Available Primitives

- KernelEvent — event envelope with topic, source, payload, timestamp
- MetricPoint — single telemetry observation type (in ume_core)
- HealthStatus — component health state model (in ume_core)
- HTTP operational endpoints: /api/v1/health, /api/v1/metrics, /api/v1/system/overview

15.2 Target Event Taxonomy

Event Topic	Trigger

kernel.boot	Kernel instantiation complete
kernel.bootstrap.completed	bootstrap_org() successful
kernel.module.registered	Module added to registry
chombo.entity.registered	Entity registered in Chombo
chombo.entity.evaluated	Entity compliance evaluation completed
soko.campaign.registered	Campaign registered in Soko
soko.signal.ingested	Market signal ingested
soko.campaign.evaluated	Campaign strategy evaluation completed
security.rbac.denied	RBAC permission check returned deny

15.3 Minimum Telemetry Requirements

- Kernel boot and bootstrap outcomes
- Subsystem evaluation request counts and durations
- Counts of generated alerts / insights / actions / filings per evaluation
- RBAC denial counts with subject and permission context

16. HTTP Server & Client Interface

ume_server exposes REST endpoints for external clients (Java GUI/CLI programs, generic HTTP clients). It bridges the HTTP boundary to the kernel facade.

16.1 Operational Endpoints

Endpoint	Description
GET /api/v1/health	Quick server/runtime status and key config/cluster info
GET /api/v1/metrics	Connection/request counters and server tuning metrics
GET /api/v1/server/clients	Request count breakdown by client ID
GET /api/v1/server/lifecycle	Current running state and connection counters
GET /api/v1/system/overview	Kernel, components, domain-modules, subsystems status
POST /api/v1/server/shutdown	Returns 202; process exits gracefully
POST /api/v1/server/cycle	Returns 202; process exits for supervisor restart

16.2 HRIS (Impande) Endpoints

Endpoint	Description
GET /api/v1/impande/summary	Impande subsystem summary
POST /api/v1/impande/workspaces/register	Register HRIS workspace
POST /api/v1/impande/workspaces/{id}/employees/upsert	Upsert employee record
POST /api/v1/impande/workspaces/{id}/benefits/upsert	Upsert benefit enrollment
POST /api/v1/impande/workspaces/{id}/reviews/upsert	Upsert performance review
POST /api/v1/impande/workspaces/{id}/timekeeping/upsert	Upsert timekeeping entry
POST /api/v1/impande/workspaces/{id}/tax-profiles/upsert	Upsert tax profile
GET /api/v1/impande/workspaces/{id}/evaluate	Evaluate HRIS workspace

16.3 Java Clients

Nine Java client programs are provided, each targeting a specific domain: chombo_client (legal), moyo_client, eneo_client, huli_client, impande_client (HRIS), imali_client (finance), pembe_client, soko_client (marketing), uso_client. Clients support three run modes:

- --gui: GUI mode with interactive panels
- --shell: interactive CLI for command-by-command operation
- --watch --interval-ms N: continuous polling and display mode

17. Architectural Risks & Technical Debt

Component	Status	Notes
No persistence for business-critical data	HIGH	All state lost on restart; production blocker
RBAC not enforced at facade boundaries	HIGH	Kernel methods bypass permission checks
Event bus not integrated into domain call paths	MED	Audit trail incomplete; observability gap
No integration test suite in CI	MED	Regressions in generated code not detectable
Generated code volume hides regressions	MED	300+ generated files require tooling support
No async execution model	LOW	Required for distributed phase; not blocking current milestone
Coarse-grained subsystem locking	LOW	Acceptable at current scale; redesign for high concurrency

18. Evolution Roadmap

The UME architecture is designed for progressive maturity across four phases. Each phase builds on the previous without requiring breaking changes to the kernel facade.

Phase 1: Operational Hardening

- Add RBAC permission guard wrappers for all Chombo and Soko kernel facade methods.
- Integrate event publication into all domain mutation and evaluation call paths.
- Add structured logging (tracing/log crate) and metrics counters throughout the kernel.
- Add CI gates: cargo fmt, cargo clippy, cargo test, integration smoke test.
- Pin test suite: implement all test cases defined in the low-level specification.

Phase 2: Durability

- Introduce persistent repository adapters implementing LegalEntityRepository, FilingRepository, CampaignRepository, and SignalRepository.
- Add schema migration scripts and schema governance process.
- Add crash recovery semantics: kernel restart must restore domain state from durable store.
- Implement event bus durable append log adapter.

Phase 3: Distributed Kernel Services

- Replace in-memory event bus with a pluggable broker adapter (Kafka, NATS, or similar).
- Introduce async orchestrator execution model using Tokio.
- Add workload partitioning for high-volume policy/strategy pack evaluations.
- Define tenant isolation boundaries and RBAC policy versioning.

Phase 4: Domain Maturity

- Replace synthetic policy/strategy pack logic with real domain-specific compliance and analytics engines.
- Introduce tenant isolation and multi-org support.
- Add governance dashboards and operator APIs.
- Build marketplace scaffolding for third-party domain module integration.
- Add fine-grained concurrency model with lock ordering framework for multi-subsystem operations.

19. Quality Attributes

Attribute	Current State	Target State
Modularity	Strong crate-level separation; trait-based extension at kernel and domain layers	Maintain; add fine-grained module boundaries as domains mature
Extensibility	New subsystem = domain crate + kernel integration module + facade methods + runtime wiring	Plugin-style subsystem registration; dynamic module loading
Scalability	Single-process in-memory	External persistence; distributed eventing; async orchestration
Reliability	Deterministic in-process behavior; supervisor decision utility present	Restart manager; durable checkpoints; retry policy framework
Observability	Event bus and MetricPoint types available; console startup log	Structured events on all mutation/evaluation paths; metrics dashboard
Security	In-memory RBAC model defined	RBAC enforced on all boundaries; authn at server layer; audit log
Testability	Unit-testable repository and rule logic	Full contract, serialization, and property test suites; CI integration

20. Testing Requirements

20.1 Required Test Layers

Layer	Coverage
Unit tests	Repository behavior; rule/policy evaluation logic; ID and timestamp invariants
Contract tests	Kernel facade methods: register + evaluate for each subsystem
Serialization tests	All command/event payload round-trips (Serialize -> Deserialize)
Property tests	ID format invariants; timestamp invariants; numeric edge cases
Integration tests	Full ume_runtime boot sequence; end-to-end domain operation flows

20.2 Minimum Required Test Cases

- Register + evaluate an existing legal entity
- Evaluate a missing legal entity (verify ENTITY-NOT-FOUND soft alert behavior)
- Register + evaluate a campaign with signals present
- Register + evaluate a campaign without signals
- RBAC permission allow path (known subject, matching permission)
- RBAC permission deny path (known subject, missing permission; unknown subject)
- Duplicate module registration rejection
- Kernel bootstrap produces deterministic org_id

21. Acceptance Criteria

This architecture specification is accepted when all of the following are true:

8. The workspace compiles successfully with ume_kernel, chombo, and soko integrated (cargo build --workspace passes).
9. The kernel facade exposes all documented methods for both Chombo and Soko subsystems.
10. Domain models and repository traits match the contracts specified in this document.
11. The ume_runtime boot path reports subsystem readiness at startup.
12. All test cases defined in Section 20.2 are implemented and green in CI.
13. RBAC enforcement is wired to all mutating and evaluation kernel facade methods.
14. Structured KernelEvents are emitted for every domain mutation and evaluation path.

Appendix A: Reference Documents

Document	Description
UME-ARCH-001	UME Organization/Business OS Architecture Specification v0.1.0
UME-LLS-001	UME Organization/Business OS Low-Level Software Specification v0.1.0
build-and-run.md	UME Build, Run, and Operational Guide

Appendix B: Backward Compatibility Policy

- Public kernel facade method signatures are semver-controlled.
- New fields in serialized structs must be additive only (no field removal without migration window).
- Existing DomainArea enum variants must not be renamed without a deprecation cycle.
- New kernel-managed subsystems must add a dedicated DomainArea variant.

Appendix C: Build Quick Reference

Command	Purpose
cargo check --workspace	Validate all crates compile
cargo build --workspace	Full workspace build
cargo run -p ume_runtime -- shell	Launch interactive CLI
cargo run -p ume_server -- --config config/ume_server.toml	Start HTTP server
cargo run -p ume_server -- health --base-url http://127.0.0.1:8080	Check server health
bazel build //clients/java:ume_java_clients	Build all Java clients
bazel run //clients/java:impande_client -- --base-url http://127.0.0.1:8080 --shell	Run Impande HRIS client
cd clients/java && mvn -q test	Run Java client tests
cargo clean && cargo check --workspace	Full clean rebuild